



Ingeniería de Software

Departamento de Ciencias de la Computación

Computación Gráfica

Proyecto Unidad 2

Título del Trabajo:

Reproductor de Música Interactivo con Visualizaciones de Espectro de Audio en Tiempo Real

Integrantes:

Mateo Llumigusin

Josue Chango

Michael Simbaña

Link: https://github.com/Josue-Chango/Proyecto_Grafica.git

Docente: Ing. Dario Morales

Fecha: 14 de junio de 2026

Índice

1. Introducción	4
2. Arquitectura y Componentes del Sistema	5
2.1. Descripción General	5
2.2. Flujo de Datos	6
3. Descripción de las Transformaciones Implementadas	6
3.1. Suavizado Temporal (EMA)	6
3.2. Detección de Beats y Pulsos	6
3.3. Física de Emisión de Partículas	7
3.4. Trigonometría de Polígonos y Transformaciones Radiales	7
4. Análisis Técnico del Procesamiento de Audio	8
4.1. Captura y Procesamiento FFT	8
4.2. Sincronización de Threads	8
5. Interfaz de Usuario y Funcionalidades	9
5.1. Controles Principales	9
5.2. Temas de Colores	9
6. Capturas de Pantalla y Evidencia de Funcionamiento	10
6.1. Interfaz Principal	10
6.2. Visualización de Barras (Tema Neón)	11
6.3. Visualización de Círculos	12
6.4. Visualización de Polígonos	13
6.5. Detección de Beats y Partículas	14
6.6. Evidencia de Funcionalidad de Controles	15
7. Código Fuente Relevante	16
7.1. Suavizado de Señal (Filtro EMA)	16
7.2. Detección de Beats (Derivada Discreta)	17
7.3. Generación y Física de Partículas	19
7.4. Renderizado Geométrico y Transformaciones Radiales	21
8. Implementación de Control de Reproducción	24
8.1. Gestión de Windows Media Player	24
8.2. Eventos y Callbacks	24

9. Conclusiones	25
10.Recomendaciones	26
10.1. Mejoras en Rendimiento y Hardware Gráfico	26
10.2. Expansión de Funcionalidades del Reproductor	26
11.Glosario de Términos Técnicos	27

1. Introducción

El diseño del reproductor de música se fundamenta en una interfaz inmersiva y moderna, priorizando el contraste visual y la experiencia de usuario. Se emplea una paleta de colores basada en un tema oscuro completo, utilizando un fondo #1C1C1C, controles en #3C3C3C y un lienzo de dibujo (canvas) completamente negro. Esta decisión estética garantiza que los elementos visuales dinámicos destaquen de manera óptima durante la reproducción.

Para la representación visual del espectro de audio, el sistema ofrece una alta personalización mediante tres temas de color seleccionables: Neón (transiciones de verde lima a magenta/cian), Azules fríos (azul claro a azul medio) y Naranjas cálidos (naranja a rojo), sumado a un efecto arcoíris rotatorio HSL para geometrías complejas.

La interacción es intuitiva, permitiendo al usuario alternar entre tres estilos de visualización principales mediante botones dedicados en la interfaz:

- **Barras:** Un conjunto de 64 barras verticales estilo ecualizador clásico, enriquecidas con degradados y un borde blanco brillante superior para remarcar los picos de frecuencia.
- **Círculos:** Un sistema radial compuesto por un núcleo brillante, 34 anillos concéntricos pulsantes, 40 rayos y anillos laterales, todo altamente reactivo al volumen y los golpes de ritmo (beats).
- **Polígonos:** Seis capas de polígonos superpuestos que sufren una deformación orgánica dictada por el espectro de frecuencias, acompañados de colores psicodélicos en rotación continua.

La disposición de los controles se rige por principios de usabilidad, organizándose en una franja horizontal inferior. A la izquierda se ubica el nombre del archivo en modo lectura ; en el centro, los controles de reproducción destacan jerárquicamente con un botón principal de Play/Pause de 74×75 píxeles, flanqueado por los botones de Stop, Anterior, Siguiente y Bucle (Loop). A la derecha, se agrupan las utilidades de volumen (TrackBar y Silenciar) y la carga de archivos. La navegación temporal se facilita mediante una barra de progreso interactiva (seeking por clic) y etiquetas de tiempo precisas.

Técnicamente, la retroalimentación visual se apoya en el renderizado sobre GDI+ a una tasa estable de 25 cuadros por segundo (FPS). El dinamismo se potencia mediante un sistema de partículas físicas que emite entre 6 y 12 partículas desde el centro en cada beat detectado, aplicando fricción y desvanecimiento. Toda esta estructura gráfica se alimenta de un análisis FFT (Transformada Rápida de Fourier) en tiempo real gestionado a través

de la biblioteca NAudio, contando además con una simulación sintética de respaldo en caso de ausencia de señal de audio.

2. Arquitectura y Componentes del Sistema

2.1. Descripción General

El reproductor de música está compuesto por varios componentes clave que trabajan en conjunto:

Cuadro 1: Componentes principales del proyecto

Componente	Descripción
frmMusicPlayer.cs	Formulario principal (interfaz de usuario) que gestiona eventos, controles y la lógica de reproducción.
CMediaPlayer.cs	Encapsulador del reproductor de Windows Media Player (COM object WMPLib). Maneja play, pause, stop y búsqueda de pistas.
AudioAnalyzer.cs	Captura de audio PCM en tiempo real mediante NAudio. Realiza análisis FFT para obtener el espectro de frecuencias en 64 bandas.
SpectrumData.cs	Gestiona los datos del espectro suavizado mediante filtro EMA. Proporciona simulación de respaldo cuando no hay audio.
BeatDetector.cs	Detecta pulsos rítmicos (beats) mediante análisis de derivada discreta de energía. Calcula vibración del centro y amplitud de pulso.
ParticleEngine.cs	Sistema de partículas física 2D. Emite partículas en beats, aplica fricción, expansión y desvanecimiento.
BarVisualization.cs	Dibuja 64 barras verticales reactivas con degradados de color según tema seleccionado.
CircleVisualization.cs	Renderiza anillos concéntricos, rayos radiales, núcleo brillante y chispas sincronizadas con audio.
PolygonVisualization.cs	Genera polígonos deformables en 6 capas con colores HSL psicodélicos.
ColorUtils.cs	Utilidades para mezcla de colores, conversión HSL-RGB y manejo de transparencia.

2.2. Flujo de Datos

Flujo de procesamiento de audio y visualización

Captura PCM (44.1 kHz) → FFT (1024 bins) → Consolidación (64 bandas) → Filtro
EMA → Detección de Beats → Visualización (GDI+) @ 25 FPS

3. Descripción de las Transformaciones Implementadas

3.1. Suavizado Temporal (EMA)

Para evitar fluctuaciones erráticas en la visualización, se implementa un filtro de primer orden de respuesta infinita al impulso (IIR) conocido como Media Móvil Exponencial (EMA). Este suavizado se aplica de forma independiente a cada una de las 64 bandas de frecuencia mediante la ecuación:

$$spectrumSmooth[i] = spectrumSmooth[i] \times \alpha + spectrum[i] \times (1 - \alpha) \quad (1)$$

Donde el factor de suavizado es $\alpha = 0,6$. Esto asigna un peso mayor al valor histórico frente al nuevo dato capturado, logrando una constante de tiempo $\tau \approx 1/(1 - \alpha)$ que equivale a 2.5 frames (aproximadamente 100ms a 25 FPS). Este enfoque garantiza una transición fluida en las barras y geometrías, operando de manera idéntica tanto con datos reales del micrófono como con la simulación sintética de respaldo.

3.2. Detección de Beats y Pulsos

El sistema de reconocimiento de ritmo no utiliza un umbral de amplitud absoluto, sino que analiza la derivada discreta (cambios bruscos de energía) de la media de amplitud entre todas las bandas:

$$\Delta = avg_actual - avg_anterior \quad (2)$$

Cuando $\Delta > 0,06$, el sistema registra un “beat”. Para asegurar un impacto visual proporcional, los golpes consecutivos acumulan fuerza hasta un máximo mediante la función:

$$BeatPulse = \text{mín}(2,0, BeatPulse + \Delta \times 8) \quad (3)$$

Este pulso decae un 8% por cada frame ($BeatPulse \times 0,92$) hasta estabilizarse en 0.01. Para evitar la saturación visual, se impone un periodo de enfriamiento (cooldown)

de 6 frames ($\sim 240\text{ms}$) entre la generación de eventos de partículas. Además, las frecuencias graves (*LowEnergy*) modulan una vibración en el núcleo central utilizando funciones trigonométricas continuas:

$$X = \sin(\text{offset} \times 0,02) \times \text{LowEnergy} \times 28$$

$$Y = \cos(\text{offset} \times 0,017) \times \text{LowEnergy} \times 18$$

3.3. Física de Emisión de Partículas

Cada pulso detectado desencadena la emisión de entre 6 y 12 partículas, limitadas a un máximo de 220 simultáneas. La inicialización y actualización de cada partícula sigue un modelo físico bidimensional:

- **Dirección y Velocidad:** Se asigna un ángulo aleatorio uniforme $\text{ang} \in [0, 2\pi)$ y una velocidad base modulada por la intensidad del beat.
- **Vectores de Movimiento:** $VX = \cos(\text{ang}) \times \text{speed}$ y $VY = \sin(\text{ang}) \times \text{speed}$.
- **Fricción y Expansión:** En cada iteración, la partícula pierde un 2 % de su velocidad (VX y $VY \times 0,98$) y su radio de dibujado se expande un 2 %.
- **Ciclo de Vida:** La opacidad (Alpha) inicia en 200 y decae a razón de 8 unidades por frame, resultando en una vida útil de aproximadamente 1 segundo antes de ser eliminada de la memoria.

3.4. Trigonometría de Polígonos y Transformaciones Radiales

La representación visual avanzada se divide en deformaciones poligonales y sistemas de anillos concéntricos:

- **Polígonos Orgánicos:** Se renderizan 6 capas superpuestas. La cantidad de lados de cada polígono es dinámica: $\text{sides} = 5 + (\text{linesToDraw} \bmod 6) + \text{layer}$. La posición angular de cada vértice incorpora un desfase continuo para rotación y un desplazamiento en cascada por capa: $\text{ang} = (s \times 2\pi / \text{sides}) + \text{offset} \times 0,02 + \text{layer} \times 0,1)2$. El radio de cada vértice se expande emparejando su índice con una banda específica del espectro suavizado (*spectrumSmooth*), aplicando un modificador orgánico 'mod' dependiente del beat y la capa, determinando su coordenada cartesiana final: $rx = \text{radiusLayer} \times \text{mod} \times \cos(\text{ang})$ y $ry = \text{radiusLayer} \times \text{mod} \times \sin(\text{ang})$.
- **Sistemas Radiales:** La visualización circular principal consiste en 34 anillos. El radio individual de cada anillo (rad) combina una progresión lineal con una oscilación sinusoidal impulsada por la energía de audio y un desplazamiento de fase escalonado,

creando un efecto de onda expansiva: $rad = baseRadius + t \times (radius \times 0,9) + \sin((offset + i \times 8) \times 0,07) \times avg \times 28 \times pulse$.

- **Elementos Complementarios:** El núcleo brillante central genera un efecto de halo mediante 6 círculos con opacidad escalonada decreciente. Las 32 chispas radiales integran un término de valor absoluto $|\cos(i + offset \times 0,03)|$ en su cálculo de longitud para simular un comportamiento de “respiración” rítmica sincronizada.

4. Análisis Técnico del Procesamiento de Audio

4.1. Captura y Procesamiento FFT

El módulo `AudioAnalyzer.cs` implementa la captura de audio PCM mediante la biblioteca NAudio, con una arquitectura que soporta múltiples escenarios:

- **Captura mediante WaveInEvent:** Callback a 44.1 kHz con buffer de 2048 muestras (≈ 46 ms).
- **Ventana de Hann:** Aplicada para reducir leakage espectral en los cálculos FFT.
- **Cálculo FFT:** NAudio.Dsp proporciona el análisis de 1024 bins de magnitud.
- **Consolidación:** Los 1024 bins se mapean a 64 bandas manejables, agrupando frecuencias relacionadas.

4.2. Sincronización de Threads

Un aspecto crítico es la protección de recursos compartidos entre:

- Hilo de callbacks de NAudio (captura de audio)
- Hilo principal de la UI (Windows Forms)
- Hilo de renderizado GDI+ (25 FPS)

Se implementa mediante directivas `lock` para garantizar consistencia en la lectura/escritura del array de espectro.

5. Interfaz de Usuario y Funcionalidades

5.1. Controles Principales

La interfaz se organiza en varias secciones:

Cuadro 2: Controles disponibles en la interfaz

Control	Función	Atajo
Play/Pause	Inicia o pausa la reproducción	Espacio / Clic
Stop	Detiene la reproducción y reinicia a cero	S
Anterior	Salta a la canción anterior	A
Siguiente	Salta a la siguiente canción	D
Loop	Activa/desactiva repetición de canción	L
Volumen (TrackBar)	Ajusta volumen 0-100 %	Rueda ratón
Mute/Unmute	Silencia/restaura volumen	M
Cargar Archivo	Abre diálogo para seleccionar MP3	Ctrl+O
Estilo Barras	Cambia visualización a barras verticales	1
Estilo Círculos	Cambia visualización a anillos circulares	2
Estilo Polígonos	Cambia visualización a polígonos deformables	3
Barra de Progreso	Permite seeking (búsqueda) dentro de la pista	Clic directo

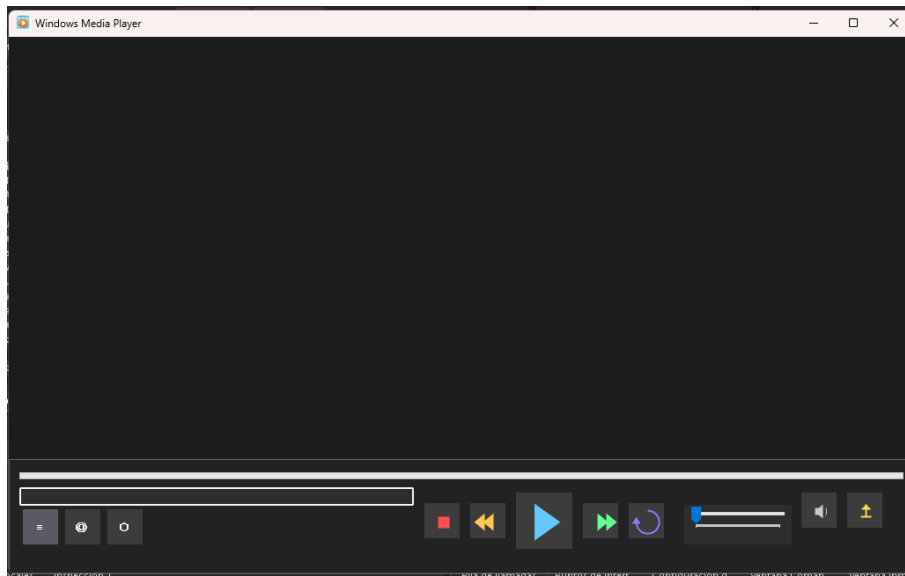
5.2. Temas de Colores

Se implementan tres esquemas de color que pueden combinarse con cualquier visualización:

1. **Tema Neón:** Gradiente de verde lima (#00FF00) a magenta/cian, proporcionando un aspecto de videojuego retro de los años 80.
2. **Tema Azules Fríos:** Transición suave desde azul claro (#0099FF) hasta azul marino (#0033FF), ideal para ambientes profesionales.
3. **Tema Naranjas Cálidos:** Gradiente de naranja vibrante (#FF6600) a rojo profundo (#CC0000), transmitiendo energía y dinamismo.
4. **Tema Arcoíris (Polígonos):** Rotación continua de colores HSL a través del espectro completo, generando efecto psicodélico.

6. Capturas de Pantalla y Evidencia de Funcionamiento

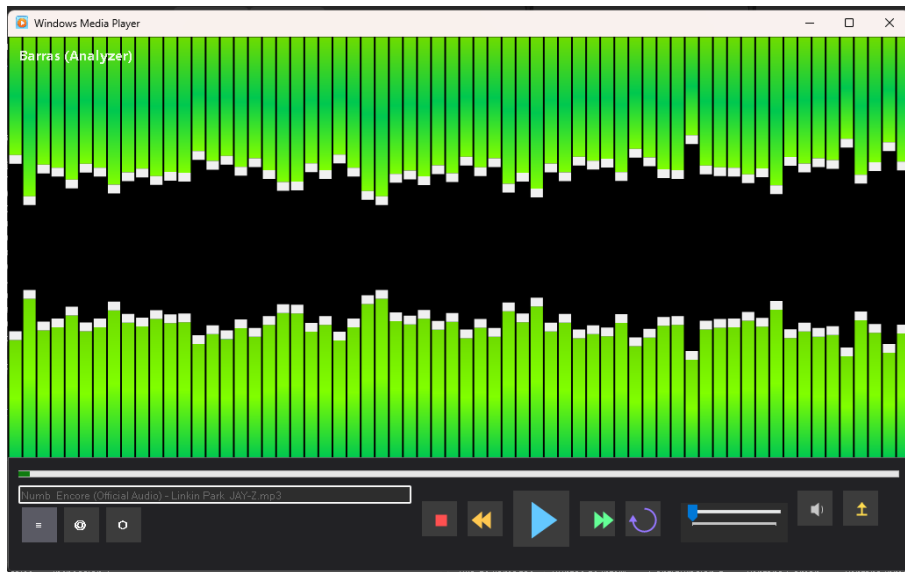
6.1. Interfaz Principal



- Área de visualización principal (lienzo negro) con anillos/barras/polígonos
- Panel de controles inferior con botones Play, Pause, Stop, Forward, Back
- Nombre del archivo cargado
- Barra de progreso indicando tiempo actual / duración total
- Control de volumen con TrackBar
- Botones para cambiar tema de color y estilo de visualización

Figura 1: Interfaz principal del reproductor de música con tema oscuro

6.2. Visualización de Barras (Tema Neón)

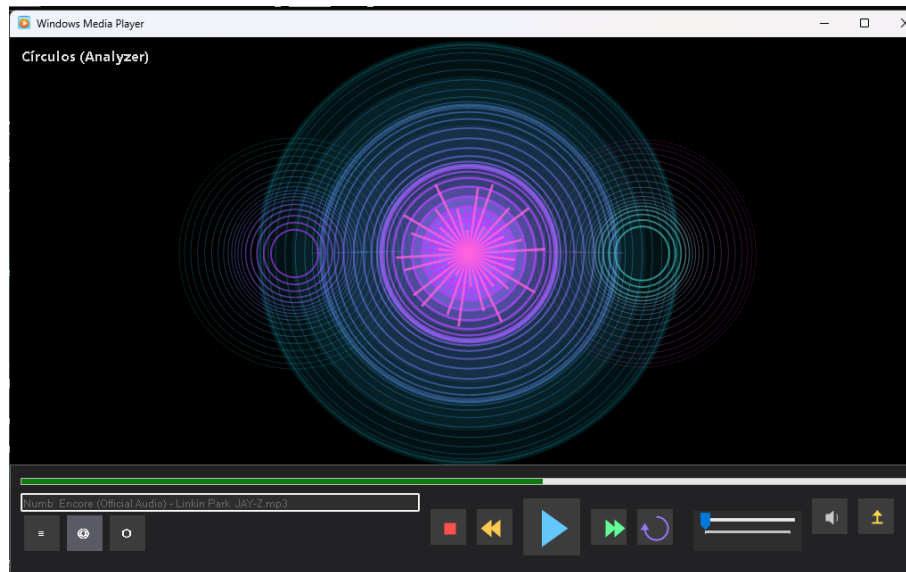


Características visibles:

- 64 barras verticales distribuidas uniformemente en el ancho del canvas
- Degradado de colores: verde lima (#00FF00) en bajos a cian (#00FFFF) en altos
- Borde blanco brillante en la parte superior de cada barra (pico de frecuencia)
- Compresión dinámica de rango: valores bajos amplificados, altos atenuados
- Suavizado EMA evitando parpadeo o vibración
- Reactividad en tiempo real a cambios en el espectro de audio

Figura 2: Representación visual mediante 64 barras con tema Neón

6.3. Visualización de Círculos

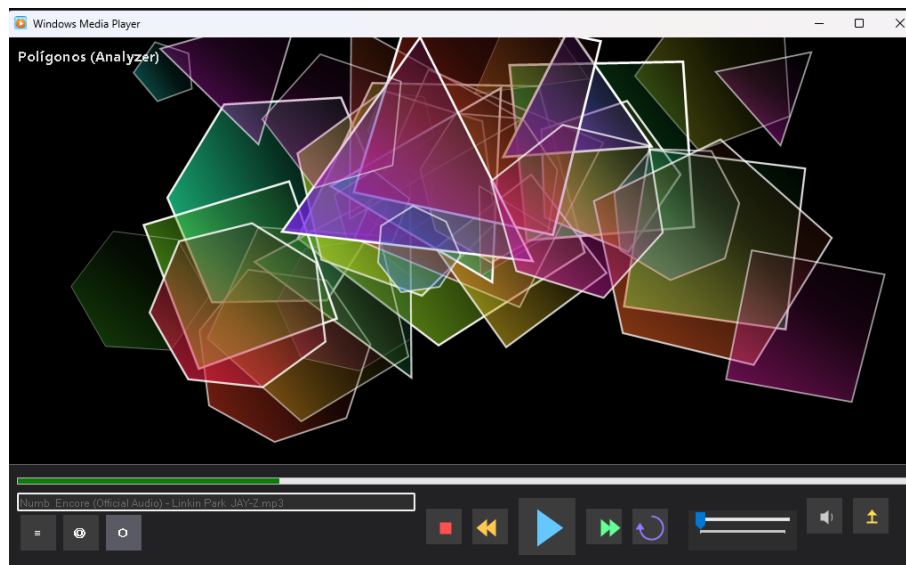


Componentes observables:

- Núcleo brillante central con efecto halo (6 capas concéntricas)
- 34 anillos concéntricos oscilando sincronizados con el audio
- 40 rayos radiales extendiéndose desde el centro, modulados por frecuencias bajas
- Partículas físicas siendo emitidas en cada beat detectado
- Vibración del núcleo causada por energía de bajos (bass shake)
- Colores degradados según tema seleccionado (Neón / Azules / Naranjas)

Figura 3: Visualización circular reactiva con sistema de partículas

6.4. Visualización de Polígonos

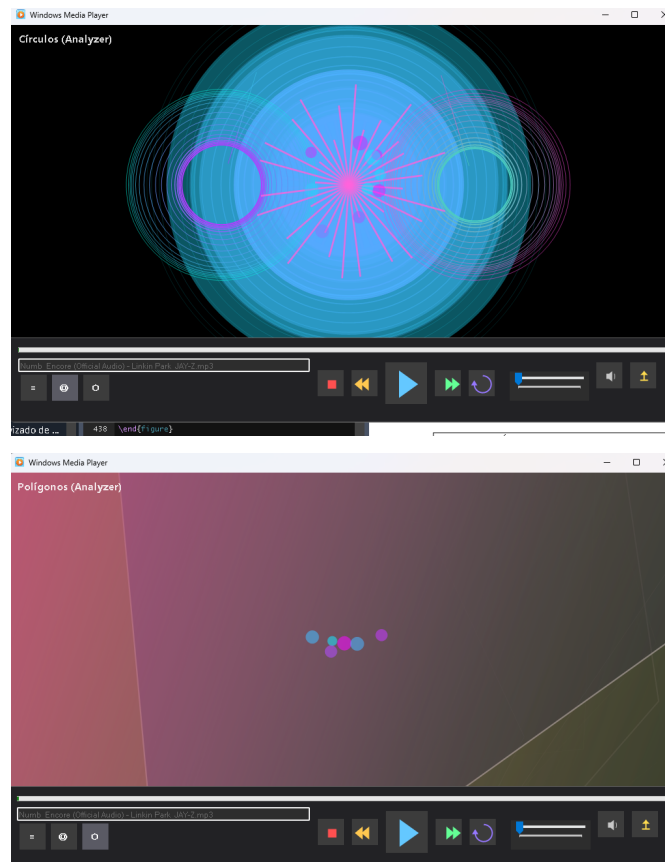


Efectos evidentes:

- 6 capas de polígonos superpuestos, cada una con número variable de lados
- Deformación orgánica en tiempo real de los vértices según espectro FFT
- Rotación continua de todos los polígonos alrededor del centro
- Colores psicodélicos rotando en HSL durante la reproducción
- Reactividad especial a beats: expansión y contracción
- Efecto de “respiración” donde los polígonos se expanden con sonido fuerte

Figura 4: Capas de polígonos deformables con efectos psicodélicos

6.5. Detección de Beats y Partículas

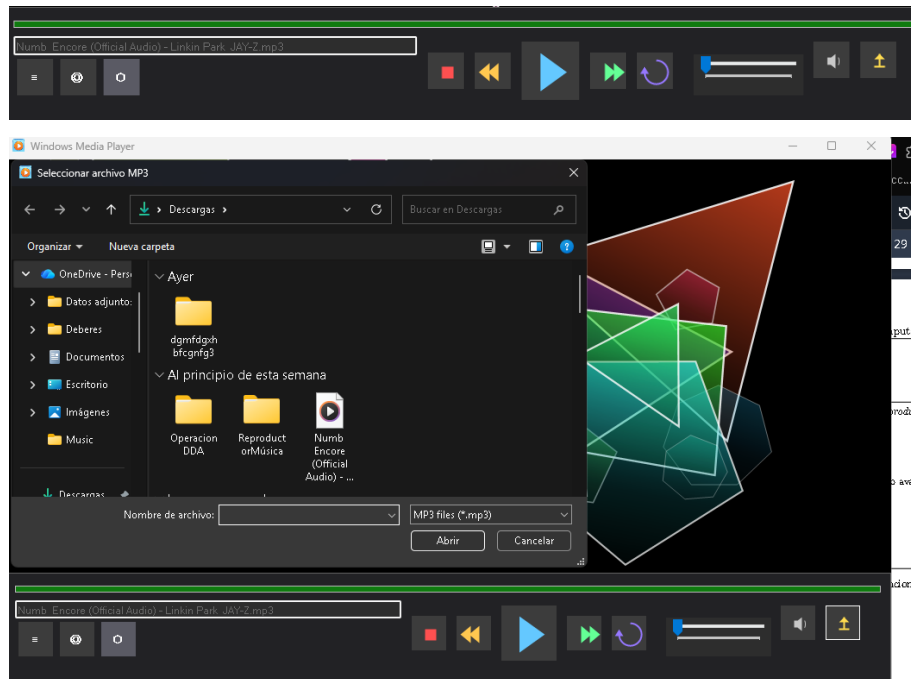


Fenómeno a observar:

- Lluvia de partículas emanando desde el centro en cada beat detectado
- Entre 6-12 partículas por evento, limitadas a máximo 220 simultáneas
- Movimiento radial en todas direcciones (ángulo aleatorio)
- Desvanecimiento gradual (fade-out) mientras se expanden
- Fricción aplicada: las partículas ralentizan un 2% por frame
- Sincronización visual clara con pulsos de ritmo en la música

Figura 5: Sistema de partículas físicas activo durante detección de beats

6.6. Evidencia de Funcionalidad de Controles



Captar fotogramas que muestren:

1. Archivo cargado pero detenido (Play button disponible)
2. Durante reproducción (Pause button disponible, barra de progreso avanzando)
3. Pausa en el medio de la canción
4. Tiempo total y tiempo transcurrido correctamente mostrados
5. Volumen variando desde mute hasta 100 %

Figura 6: Secuencia de estados de reproducción y controles funcionales

7. Código Fuente Relevante

7.1. Suavizado de Señal (Filtro EMA)

En esta sección, se implementa la Media Móvil Exponencial para evitar cambios bruscos en la lectura de las bandas de frecuencia, logrando una transición visual fluida.

```
1 // SpectrumData.cs - Gestiona los datos del espectro de audio para la
  visualizaci n
2 using System;
3
4 namespace ReproductorM sica
5 {
6     internal class SpectrumData
7     {
8         private int barCount = 64;
9         private float[] spectrum;
10        private float[] spectrumSmooth;
11        // Factor de suavizado EMA: 0 = sin suavizar, 1 = completamente
        suave
12        private float spectrumSmoothing = 0.6f;
13        private Random rng = new Random();
14        private AudioAnalyzer analyzer;
15
16        public float[] SpectrumSmooth => spectrumSmooth;
17
18        public SpectrumData(AudioAnalyzer analyzer = null)
19        {
20            this.analyzer = analyzer;
21            spectrum = new float[barCount];
22            spectrumSmooth = new float[barCount];
23        }
24
25        // Actualiza el espectro cada frame aplicando el filtro EMA
26        public void Update(double position, int offset)
27        {
28            if (analyzer != null)
29            {
30                try
31                {
32                    var real = analyzer.GetSpectrum(barCount);
33                    for (int i = 0; i < barCount && i < real.Length; i++)
34                    {
35                        spectrum[i] = real[i];
36                        // Filtro EMA: suaviza la transici n entre el valor
```



```

37         // hist rico y el nuevo dato capturado
38         spectrumSmooth[i] = spectrumSmooth[i] * spectrumSmoothing
39             + spectrum[i] * (1f - spectrumSmoothing);
40     }
41 }
42 catch { }
43 }
44
45 // Simulaci n matem tica de respaldo (sin micr fono)
46 int seed = (int)(DateTime.Now.Ticks & 0xFFFFFFFF);
47 var localRng = new Random(seed ^ offset);
48 for (int i = 0; i < barCount; i++)
49 {
50     double t = position * 2.0 + i * 0.13 + offset * 0.01
51         + localRng.NextDouble() * 0.5;
52     float value = (float)((Math.Abs(Math.Sin(t + i)) * 0.8)
53         + (localRng.NextDouble() * 0.2));
54     float falloff = 1.0f - (i / (float)barCount);
55     value *= (0.3f + 0.7f * falloff);
56     if (analyzer == null)
57     {
58         spectrum[i] = value;
59         spectrumSmooth[i] = spectrumSmooth[i] * spectrumSmoothing
60             + spectrum[i] * (1f - spectrumSmoothing);
61     }
62 }
63 }
64
65 public void Reset()
66 {
67     for (int i = 0; i < spectrum.Length; i++)
68         spectrum[i] = spectrumSmooth[i] = 0f;
69 }
70 }
71 }

```

Listing 1: SpectrumData.cs - Gestión del espectro suavizado

7.2. Detección de Beats (Derivada Discreta)

El siguiente fragmento muestra la lógica para detectar golpes de ritmo basándose en cambios bruscos de energía en lugar de umbrales estáticos.

```

1 // BeatDetector.cs - Detecta golpes r tmicos por derivada discreta de
   energ a

```

```

2 using System;
3
4 namespace ReproductorMusica
5 {
6     internal class BeatDetector
7     {
8         private float prevAvg = 0f;    // Promedio del frame anterior
9         private int beatCooldown = 0;  // Cooldown: 6 frames (~240 ms)
10        public float BeatPulse { get; private set; } // Pulso acumulado
11        (0..2)
12        public float LowEnergy { get; private set; } // Energía de graves
13        public float CenterOffsetX { get; private set; }
14        public float CenterOffsetY { get; private set; }
15        public float Delta { get; private set; }      // Derivada discreta
16        public bool CanSpawn => beatCooldown == 0;
17
18        // Analiza el espectro para detectar beats
19        public void Analyze(float[] spectrumSmooth, int offset)
20        {
21            // 1. Promedio general de amplitud
22            float avg = 0f;
23            for (int i = 0; i < spectrumSmooth.Length; i++)
24                avg += spectrumSmooth[i];
25            avg /= spectrumSmooth.Length;
26
27            // 2. Energía de frecuencias bajas (graves)
28            int lowBands = Math.Max(2, spectrumSmooth.Length / 12);
29            float lowSum = 0f;
30            for (int i = 0; i < lowBands; i++)
31                lowSum += spectrumSmooth[i];
32            LowEnergy = lowSum / lowBands;
33
34            // 3. Vibración del centro basada en graves (bass shake)
35            CenterOffsetX = (float)(Math.Sin(offset * 0.02) * LowEnergy *
36            28.0);
37            CenterOffsetY = (float)(Math.Cos(offset * 0.017) * LowEnergy *
38            18.0);
39
40            // 4. Derivada discreta: Delta > 0.06 indica un beat
41            Delta = avg - prevAvg;
42            if (Delta > 0.06f)
43                BeatPulse = Math.Min(2.0f, BeatPulse + Delta * 8f);
44            prevAvg = avg;
45        }
46    }
47 }

```

```

44 // Activa el periodo de enfriamiento (6 frames)
45 public void ResetCooldown() { beatCooldown = 6; }
46
47 // Decaimiento del pulso: 8% por frame (factor 0.92)
48 public void Decay()
49 {
50     BeatPulse *= 0.92f;
51     if (BeatPulse < 0.01f) BeatPulse = 0f;
52     if (beatCooldown > 0) beatCooldown--;
53 }
54
55 public void Reset()
56 {
57     prevAvg = 0f; BeatPulse = 0f; beatCooldown = 0;
58 }
59 }
60 }

```

Listing 2: BeatDetector.cs - Detección de beats

7.3. Generación y Física de Partículas

Este código define cómo se instancian las partículas, asignándoles vectores de velocidad y ángulos aleatorios tras detectar un pulso.

```

1 // ParticleEngine.cs - Sistema de partículas reactivo a los beats
2 using System;
3 using System.Collections.Generic;
4 using System.Drawing;
5
6 namespace ReproductorMusica
7 {
8     internal class ParticleEngine
9     {
10         private class Particle
11         {
12             public float X, Y; // Posición actual
13             public float VX, VY; // Velocidad en cada eje
14             public float Radius; // Radio (crece con el tiempo)
15             public int Alpha; // Transparencia (200 -> 0)
16             public Color Color;
17         }
18
19         private List<Particle> particles = new List<Particle>();
20         private int maxParticles = 220; // Límite de rendimiento

```

```

21     private Random rng = new Random();
22
23     // Genera entre 6 y 12 partículas al detectar un beat
24     public void SpawnParticles(float cx, float cy,
25         float intensity, Color innerColor, Color outerColor)
26     {
27         int count = 6 + rng.Next(6);
28         for (int i = 0; i < count; i++)
29         {
30             if (particles.Count >= maxParticles) break;
31             double ang = rng.NextDouble() * Math.PI * 2.0;
32             float speed = (float)(2.0 + rng.NextDouble() * 6.0 * intensity);
33             Particle p = new Particle();
34             p.X = cx + (float)(Math.Cos(ang) * rng.NextDouble() * 8f);
35             p.Y = cy + (float)(Math.Sin(ang) * rng.NextDouble() * 8f);
36             p.VX = (float)(Math.Cos(ang) * speed);
37             p.VY = (float)(Math.Sin(ang) * speed);
38             p.Radius = (float)(2.0 + rng.NextDouble() * 4.0 + intensity *
14.0f);
39             p.Alpha = 200; // Opacidad inicial
40             p.Color = ColorUtils.BlendColors(innerColor, outerColor,
41                 (float)rng.NextDouble());
42             particles.Add(p);
43         }
44     }
45
46     // Actualiza físicas y dibuja: fricción 0.98, radio crece 1.02,
47     // alpha decrece 8 por frame (~1 s de vida total)
48     public void UpdateAndDrawParticles(Graphics g)
49     {
50         for (int i = particles.Count - 1; i >= 0; i--)
51         {
52             var p = particles[i];
53             p.X += p.VX;    p.Y += p.VY;
54             p.VX *= 0.98f; p.VY *= 0.98f; // Fricción
55             p.Radius *= 1.02f;           // Expansión
56             p.Alpha -= 8;                // Desvanecimiento
57             try
58             {
59                 int a = ColorUtils.ClampAlpha(p.Alpha);
60                 using (SolidBrush br = new SolidBrush(
61                     Color.FromArgb(a, p.Color.R, p.Color.G, p.Color.B)))
62                 {
63                     float rr = Math.Max(1f, p.Radius);
64                     g.FillEllipse(br, p.X - rr, p.Y - rr, rr * 2, rr * 2);

```

```

65     }
66     } catch { }
67     if (p.Alpha <= 0 || p.Radius > 5000) particles.RemoveAt(i);
68 }
69 }
70
71 public void Clear() { particles.Clear(); }
72 }
73 }

```

Listing 3: ParticleEngine.cs - Sistema de partículas

7.4. Renderizado Geométrico y Transformaciones Radiales

A continuación, se detalla la función principal de dibujado que deforma los vértices de los polígonos sincronizados con la señal de audio.

```

1 // CircleVisualization.cs (fragmento principal) - Anillos conc ntricos
   reactivos
2 using System;
3 using System.Drawing;
4
5 namespace ReproductorM sica
6 {
7     internal class CircleVisualization
8     {
9         public void Draw(Graphics g, int offset, float[] spectrumSmooth,
10             float beatPulse, float lowEnergy, float cx, float cy,
11             int radius, Color innerColor, Color outerColor)
12         {
13             float avg = 0f;
14             for (int i = 0; i < spectrumSmooth.Length; i++)
15                 avg += spectrumSmooth[i];
16             avg /= spectrumSmooth.Length;
17
18             float pulse      = 1f + beatPulse * 1.2f;
19             float baseRadius = radius * 0.16f * pulse * 1.25f;
20             int    rings      = 34;
21
22             // === N CLEO BRILLANTE (6 capas con opacidad decreciente) ===
23             Color coreColor = ColorUtils.BlendColors(innerColor, outerColor,
24                 Math.Max(0f, Math.Min(1f, lowEnergy)));
25             float coreBase = baseRadius * 0.28f + avg * radius * 0.18f
26                 + beatPulse * 18f;
27             for (int gstep = 5; gstep >= 0; gstep--)

```

```

28     {
29         int alpha = ColorUtils.ClampAlpha((int)(200*(1.0f - gstep/6.0f))
30     );
31         float rr = coreBase * (1.0f + gstep * 0.45f);
32         if (rr > 0.5f)
33             using (SolidBrush br = new SolidBrush(
34                 Color.FromArgb(alpha, coreColor.R, coreColor.G, coreColor.B)
35             ))
36                 g.FillEllipse(br, cx-rr, cy-rr, rr*2, rr*2);
37     }
38
39     // === 40 RAYOS DESDE EL CENTRO ===
40     using (Pen starPen = new Pen(
41         Color.FromArgb(240, innerColor.R, innerColor.G, innerColor.B),
42         Math.Max(1f, 2f + beatPulse * 3f)))
43     {
44         for (int i = 0; i < 40; i++)
45         {
46             double ang = (i * 2 * Math.PI / 40) + offset * 0.04;
47             float len = coreBase * 0.6f + avg * radius * 0.45f * pulse
48                 + beatPulse * 12f;
49             if (len > 0.5f)
50                 g.DrawLine(starPen, cx, cy,
51                     cx + (float)(Math.Cos(ang)*len),
52                     cy + (float)(Math.Sin(ang)*len));
53         }
54     }
55
56     // === 34 ANILLOS CONCENTRICOS (oscilación sinusoidal por beat)
57     ===
58     for (int i = 0; i < rings; i++)
59     {
60         float t = i / (float)rings;
61         // Radio con oscilación sinusoidal modulada por volumen y beat
62         float rad = baseRadius + t * (radius * 0.9f)
63             + (float)(Math.Sin((offset + i*8)*0.07) * avg*28*pulse);
64         int alpha = ColorUtils.ClampAlpha((int)(200 - i*(180/rings)));
65         Color c = ColorUtils.BlendColors(innerColor, outerColor, t);
66         if (rad > 0.5f)
67             using (Pen p = new Pen(
68                 Color.FromArgb(alpha, c.R, c.G, c.B),
69                 Math.Max(1f, 3f - t * 2.6f)))
70                 g.DrawEllipse(p, cx-rad, cy-rad, rad*2, rad*2);
71     }
72 }

```

```
70      // === 32 CHISPAS RADIALES (longitud con |cos| para efecto
      respiraci n) ===
71      for (int i = 0; i < 32; i++)
72      {
73          double ang = (i * 2 * Math.PI / 32) + offset * 0.03 * pulse;
74          float len  = baseRadius * 0.5f
75                      + avg * radius * 0.8f
76                      * (float)Math.Abs(Math.Cos(i + offset * 0.03)) * pulse;
77          using (Pen p = new Pen(Color.FromArgb(200, 255, 100, 220),
78          1 + (float)(Math.Abs(Math.Sin(offset * 0.05)) * 1.8f)))
79              g.DrawLine(p, cx, cy,
80                  cx + (float)(Math.Cos(ang)*len),
81                  cy + (float)(Math.Sin(ang)*len));
82      }
83  }
84  }
85 }
```

Listing 4: CircleVisualization.cs - Visualización circular

8. Implementación de Control de Reproducción

8.1. Gestión de Windows Media Player

El módulo `CMediaPlayer.cs` encapsula la interacción con Windows Media Player mediante COM (WMPLib):

```

1 // Pseudocódigo de operaciones comunes
2 Play()           // Inicia reproducción
3 Pause()          // Pausa la canción actual
4 Stop()           // Detiene y reinicia a posición 0
5 SetVolume(0-1)   // Establece volumen normalizado 0-100%
6 GetPosition()    // Retorna tiempo actual en segundos
7 SetPosition()    // Busca a posición específica (seeking)
8 GetDuration()    // Obtiene duración total de la pista
9 LoadFile(path)  // Carga archivo MP3 desde ruta

```

Listing 5: Operaciones básicas del reproductor

8.2. Eventos y Callbacks

La lógica de reproducción se maneja mediante eventos del formulario:

Cuadro 3: Eventos principales y sus triggers

Evento	Trigger / Acción
<code>btnPlayPause_Click</code>	Alterna entre Play y Pause
<code>btnStop_Click</code>	Detiene reproducción, reinicia posición
<code>btnForward_Click</code>	Salta a siguiente track (si hay lista)
<code>btnBackward_Click</code>	Salta a pista anterior
<code>timer1_Tick</code>	Cada 250ms: actualiza barra de progreso, muestra tiempo
<code>timer2_Tick</code>	Cada 40ms: refresca visualización GDI+ (25 FPS aprox.)
<code>trbVolume_Scroll</code>	Ajusta volumen del reproductor en tiempo real
<code>progressBar_MouseDown</code>	Permite seeking: usuario clickea en barra de progreso

9. Conclusiones

- **Efectividad del Suavizado Matemático:** La implementación de la Media Móvil Exponencial (EMA) con un factor de 0.6 probó ser fundamental para la experiencia visual. Sin este filtro, el espectro FFT crudo (basado en ventanas de 2048 muestras cada $\sim 46\text{ms}$) resultaba demasiado sensible al ruido, produciendo un “temblor” errático. El EMA introduce una constante de tiempo de $\sim 100\text{ms}$ que transforma este comportamiento en una respuesta fluida y orgánica, permitiendo que los elementos gráficos “respiren” rítmicamente en lugar de vibrar descontroladamente.
- **Trigonometría Aplicada al Dinamismo GDI+:** El uso de funciones trigonométricas permitió generar comportamientos complejos en la interfaz gráfica. Destaca la compresión de rango dinámico mediante potencias no lineales ($\text{pow}(\text{amp}, 0,6)$) en las barras, que garantiza visibilidad en pasajes acústicos suaves. Igualmente, las oscilaciones radiales con fases desplazadas en los anillos concéntricos y las deformaciones orgánicas de vértices en las capas poligonales demostraron la flexibilidad de GDI+ para crear ilusiones de profundidad, movimiento continuo y efectos psicodélicos.
- **Integración Exitosa de NAudio:** El pipeline de procesamiento de audio se estructuró de manera eficiente: desde la captura PCM mediante WaveInEvent a 44.1 kHz, pasando por la aplicación de una ventana de Hann para reducir el leakage espectral, hasta el cálculo FFT utilizando NAudio.Dsp. La consolidación de 1024 bins de magnitud a 64 bandas manejables optimizó significativamente el análisis en tiempo real.
- **Gestión de Hilos y Concurrencia:** Un logro técnico importante fue la sincronización y protección de recursos (mediante la directiva lock) entre el hilo que maneja los callbacks de NAudio y el hilo principal de la interfaz de usuario (UI), garantizando estabilidad operativa durante la actualización continua del lienzo GDI+ a 25 FPS sin errores de concurrencia.
- **Interactividad en Tiempo Real:** El sistema demuestra la importancia del feedback visual inmediato. La capacidad de cambiar visualizaciones (barras/círculos/-polígonos) y temas de color en vivo, junto con seeking interactivo en la barra de progreso, proporcionó una experiencia altamente respuesta y profesional.

10. Recomendaciones

10.1. Mejoras en Rendimiento y Hardware Gráfico

- **Migración de API Gráfica:** Actualmente, la limitación principal es el fill rate derivado del renderizado exclusivo por CPU mediante GDI+. Se recomienda migrar el motor visual hacia librerías que ofrezcan aceleración por GPU (como OpenGL vía OpenTK, o Direct2D vía SharpDX). Esto descargaría a la CPU, permitiendo estabilizar la tasa de fotogramas por encima de los 60 FPS, habilitando mayor densidad de partículas y resolución sin degradar el rendimiento.
- **Computación en Paralelo (Shaders):** Para futuras iteraciones, es aconsejable delegar los cálculos matemáticos intensivos—como el análisis FFT de NAudio y la física del sistema de partículas—a Compute Shaders (DirectCompute o OpenCL), procesando los datos directamente en el procesador gráfico.

10.2. Expansión de Funcionalidades del Reproductor

- **Soporte Extendido de Audio:** Sustituir la dependencia del componente Windows Media Player por una implementación nativa en NAudio que soporte formatos sin pérdida como FLAC, además de OGG y AAC. Asimismo, la implementación del protocolo WASAPI Exclusive reduciría la latencia, mejorando la sincronía audio/-visual.
- **Mejoras en la Experiencia de Usuario (UX):** Desarrollar un sistema robusto de listas de reproducción con funcionalidad Drag & Drop y persistencia de datos (JSON/XML). Adicionalmente, se sugiere incorporar atajos de teclado globales (hotkeys) para control en segundo plano y transiciones suaves (crossfade) entre pistas.
- **Nuevos Módulos Visuales:** Ampliar la interactividad permitiendo paletas de colores totalmente personalizables en tiempo real, junto con la adición de nuevos modos de visualización como espectrogramas tipo cascada clásica, soporte superpuesto para visualización de letras sincronizadas (formato LRC) y un modo inmersivo de pantalla completa.
- **Análisis Avanzado de Audio:** Incorporar algoritmos adicionales como análisis de timbre, detección automática de género musical, beatgridding preciso para producción musical, y extracción de características (MFCC) para retroalimentación visual más sofisticada.

11. Glosario de Términos Técnicos

FFT Transformada Rápida de Fourier. Algoritmo que descompone una señal de audio en sus componentes de frecuencia.

EMA Media Móvil Exponencial. Filtro IIR que suaviza datos históricos dando mayor peso a valores recientes.

GDI+ Graphics Device Interface Plus. API de Windows para renderizado 2D de gráficos.

NAudio Biblioteca .NET de código abierto para procesamiento de audio.

Beat Pulso rítmico detectado mediante análisis de cambios de energía en el espectro.

Cooldown Período de espera para evitar desencadenamiento excesivo de eventos (e.g., emisión de partículas).

WaveInEvent Dispositivo virtual de NAudio para captura de audio en tiempo real.

HSL Hue-Saturation-Lightness. Modelo de color alternativo a RGB.

Leakage Espectral Distorsión en cálculos FFT causada por discontinuidades; mitigada con ventanas (Hann).

Seeking Acción de saltar a una posición específica dentro de una pista de audio.

Opacidad / Alpha Grado de transparencia de un color (0 = totalmente transparente, 255 = opaco).